



# **Welke beveiligingsproblemen komen voor bij REST-API's en hun authenticatiesystemen, en hoe kunnen deze worden voorkomen?**

2025 – 2026  
GDC – 3<sup>De</sup> Graad

Ian-Chains Baute – 6ICWE  
Generieke Doorstroom Competentie  
Informaticawetenschappen  
Technisch Onderzoek

GDC Mentor: J. Gillis

## Woord vooraf

Digitale systemen en softwarepakketten worden steeds groter, uitgebreider en complexer maar de veiligheid en beveiliging mag niet vergeten worden, deze beveiliging wordt ook steeds uitdagender en complexer samen met de groei van de software. Uitgebreide software systemen maken vaak gebruik van API's om data uit te wisselen tussen de gebruiker of andere software systemen en daarbij is de REST-API de meest voorkomende en gebruikte.

Ik heb zelf al meerdere keren REST-API's gebruikt in projecten voor het uitwisselen van data tussen de gebruiker en de app. Maar ik heb zelf nog nooit echt stil gestaan bij de beveiliging van de API voor mijn projecten. Daarom wil ik in dit onderzoek dan ook de nadruk leggen op de beveiliging van een REST-API systeem om bij te leren over welke beveiligingsproblemen er vaak aanwezig zijn en hoe ze voorkomen kunnen worden.

Dit technisch onderzoek over de beveiliging van REST-API's heb ik uitgevoerd als voorbereiding op een vervolg project waarbij ik deze informatie over de beveiliging kan gebruiken om op een veilige manier een API systeem op te zetten bij mijn vervolg projecten.

Graag wil ik een aantal mensen vermelden en bedanken voor hun assistentie bij dit GDC onderzoek. Ik wil in het bijzonder mijn mentor meneer Gillis bedanken voor de steun en tips tijdens dit onderzoek.

# Inhoudsopgave

|                                                       |    |
|-------------------------------------------------------|----|
| Woord vooraf.....                                     | 1  |
| Inleiding.....                                        | 3  |
| Afkortingen.....                                      | 4  |
| 1 XSS - Cross-Site Scripting .....                    | 5  |
| 1.1 Wat is XSS?.....                                  | 5  |
| 1.1.1 Algemeen.....                                   | 5  |
| 1.1.2 Reflected XSS.....                              | 6  |
| 1.1.3 Stored XSS .....                                | 6  |
| 1.2 Hoe XSS voorkomen?.....                           | 7  |
| 1.2.1 Algemeen.....                                   | 7  |
| 1.2.2 HTML escaping - HTML entity encoding.....       | 7  |
| 1.2.3 HTML Sanitization & HTML Stripping .....        | 7  |
| 1.2.4 Cookies met HttpOnly attribuut.....             | 8  |
| 1.3 Praktisch.....                                    | 9  |
| 1.3.1 Reflected XSS bij onveilige API.....            | 9  |
| 1.3.2 Stored XSS bij onveilige API .....              | 10 |
| 1.3.3 Veilige API beschermt tegen XSS .....           | 12 |
| 2 CSRF - Cross-Site Request Forgery .....             | 13 |
| 2.1 Wat is CSRF? .....                                | 13 |
| 2.1.1 Algemeen.....                                   | 13 |
| 2.1.2 Voorbeeld Normaal Request .....                 | 13 |
| 2.1.3 Voorbeeld CSRF .....                            | 14 |
| 2.2 Hoe CSRF voorkomen? .....                         | 15 |
| 2.2.1 SOP & CORS.....                                 | 15 |
| 2.2.2 Cookies met SameSite attribuut.....             | 15 |
| 2.2.3 (Anti-)CSRF Tokens .....                        | 16 |
| 2.3 Praktisch.....                                    | 17 |
| 2.3.1 CSRF bij onveilige API .....                    | 17 |
| 2.3.2 CSRF ondanks correcte CORS configuratie.....    | 18 |
| 2.3.3 (Anti-)CSRF Tokens beschermen veilige API ..... | 19 |
| Besluit .....                                         | 21 |
| Bijlagen .....                                        | 22 |
| Referentielijst .....                                 | 22 |

# Inleiding

In dit werk wordt er een technisch onderzoek gedaan naar API<sup>1</sup> systemen zowel theoretisch als praktisch, om de algemene beveiliging van API systemen beter te begrijpen en hierop te anticiperen. Dit onderzoek gaat dieper in op 2 veelvoorkomende beveiligingsproblemen bij REST-API's<sup>2</sup> met de focus op het uitwisselen van data tussen een gebruiker en een applicatie.

De 2 beveiligingsproblemen die onderzocht worden zijn: Cross-Site Scripting (XSS) en Cross-Site Request Forgery (CSRF). Het 1<sup>ste</sup> deel van elk beveiligingsprobleem bestaat uit een theoretisch onderzoek naar de aard van het probleem, uitgevoerd door een literatuurstudie om beter te begrijpen wat het probleem precies inhoudt.

Het 2<sup>de</sup> deel van het onderzoek bestaat uit een praktisch productie en ontwikkelingsgedeelte, hierbij worden 2 overkoepelende API systemen opgezet. Het doel en de functionaliteiten van de 2 API systemen is hetzelfde, namelijk het kunnen aanmaken en inloggen met een account en daarnaast het aanmaken en opvragen van (blog)posts.

De 2 API systemen verschillen in de vorm van onveilig en veilig, waarbij er bij het 1<sup>ste</sup> systeem geen extra acties en stappen ondernomen worden voor het voorkomen van de onderzochte beveiligingsproblemen. Bij het 2<sup>de</sup> systeem zijn deze extra acties en stappen wel ondernomen. Door beide systemen met elkaar te vergelijken en te kijken naar de extra acties die ondernomen worden voor het veilig maken van het API systeem, kan worden aangetoond welke maatregelen nodig zijn om de beveiligingsproblemen te voorkomen.

Het praktische gedeelte wordt volledig gedocumenteerd via een GitHub repository, de link naar de repository is opgenomen in de bijlagen van dit werk. Bij het praktische gedeelte is er gebruik gemaakt van Python en FastAPI om een API systeem op te zetten, meer informatie over de gebruikte technologiestack kan terug gevonden worden in de repository.

---

<sup>1</sup> Informatie over wat een API is en hoe het technisch werkt kan gevonden worden in dit artikel van Geeks For Geeks: GeeksforGeeks. (2025, December 15). *What is an API (Application Programming Interface)*. GeeksforGeeks. <https://www.geeksforgeeks.org/software-testing/what-is-an-api/>

<sup>2</sup> Informatie over wat een REST-API is en hoe het technisch werkt kan gevonden worden in dit artikel van Postman: Team, P. (2025, December 16). *What is a REST API? Examples, uses, and challenges*. Postman Blog. <https://blog.postman.com/rest-api-examples/>

## Afkorting

|        |                                   |
|--------|-----------------------------------|
| GDC    | Generieke Doorstroom Competentie  |
| i.p.v. | in plaats van                     |
| i.v.m. | in verband met                    |
| API    | Application Programming Interface |
| REST   | Representational State Transfer   |
| XSS    | Cross-Site Scripting              |
| CSRF   | Cross-Site Request Forgery        |
| SOP    | Same Origin Policy                |
| CORS   | Cross-Origin Resource Sharing     |
| DOM    | Document Object Model             |
| DB     | Database                          |
| Py     | Python                            |
| JS     | JavaScript                        |
| HTML   | HyperText Markup Language         |
| HTTP   | HyperText Transfer Protocol       |
| URL    | Uniform Resource Locator          |
| webapp | webapplicatie                     |
| UID    | Unique Identifier                 |

# 1 XSS - Cross-Site Scripting

## 1.1 Wat is XSS?

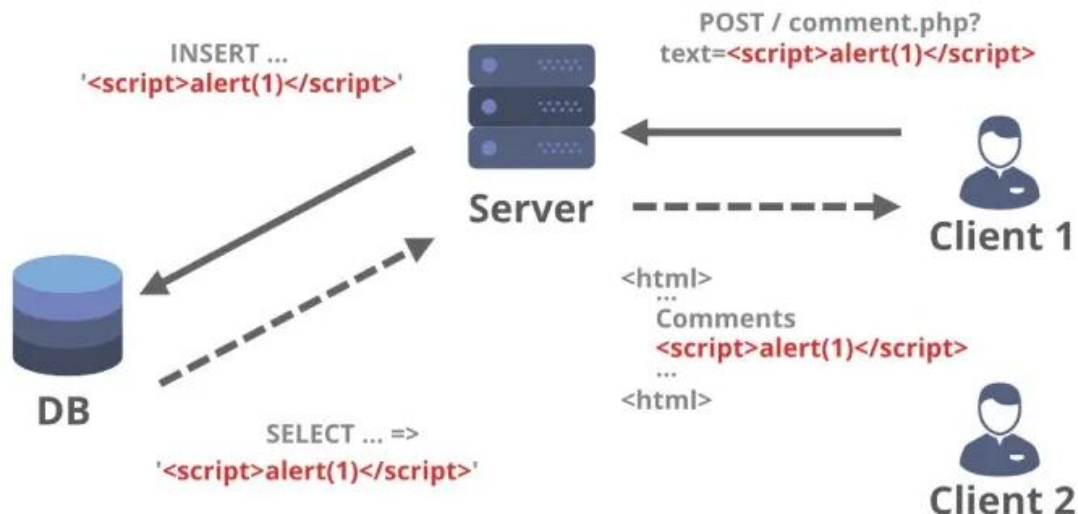
### 1.1.1 Algemeen

XSS is een kwetsbaarheid in een webapp dat iemand toelaat om JavaScript te injecteren in de webapp en die te laten uitvoeren in de browser van een andere gebruiker in naam van de vertrouwde maar kwetsbare webapplicatie. Daarbij draait dus de JavaScript code in de browser van het slachtoffer, wat een gewone gebruiker of een beheerder kan zijn, en de browser denkt daarbij dat de code afkomstig is van de vertrouwde webapp. Hierdoor kan de code gevoelige zaken uitvoeren zoals:

- (Cookies met) sessie gegevens stelen
- Zich voordoen als een andere gebruiker
- Formulieren manipuleren
- Ongewenste API requests versturen
- Content van de website aanpassen
- ...

**Figuur 1**

Visualisatie (Stored) XSS



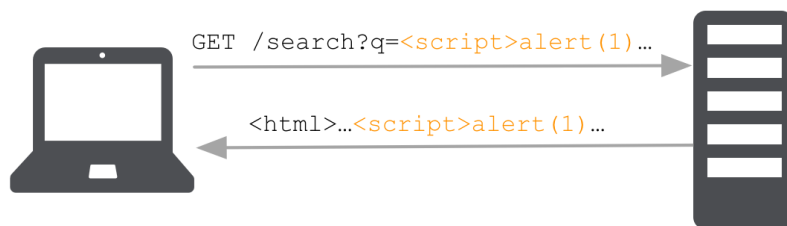
GeeksforGeeks (2025)

Figuur 1 is een visuele weergave van een XSS kwetsbaarheid die wordt getest, waarbij client 1 de aanvaller is en client 2 het slachtoffer. Er wordt in de figuur een veel gebruikte en simpele JavaScript functie genaamd alert gebruikt om de kwetsbaarheid van XSS aan te tonen. Wanneer het slachtoffer een pop-up in zijn/haar beeld krijgt, dan is er effectief sprake van een XSS kwetsbaarheid. XSS Kan opgedeeld worden in verschillende categorieën: reflected en stored XSS. Deze figuur is een voorbeeld van stored XSS want de code wordt opgeslagen in de database.

### 1.1.2 Reflected XSS

#### Figuur 2

Visualisatie reflected XSS bij zoek functie



Hepper (2017)

Bij reflected XSS wordt de code niet opgeslagen maar meteen gereflecteerd en teruggestuurd in de HTTP-response naar de browser van het slachtoffer. Een reflected XSS kwetsbaarheid komt vaak voor bij zoekfuncties, net zoals bij figuur 2. Er wordt dan JavaScript code meegegeven als zoekterm i.p.v. een realistische zoekterm en dit is exact wat er in figuur 2 gebeurt. De server in figuur 2 verwerkt het verzoek en geeft de zoekresultaten en de opgezochte term terug aan de gebruiker. Hierdoor bevindt de JavaScript zich in de HTTP-response. Doordat de code zich in de HTTP-response bevindt kan deze uitgevoerd worden door de browser van het slachtoffer en kan dit schade veroorzaken.

Een reflected XSS kwetsbaarheid stelt een aanvaller in staat om een kwaadaardige URL te maken met daarin JavaScript en deze door te sturen naar een slachtoffer. Het slachtoffer krijgt deze link en ziet dat deze naar de officiële (maar kwetsbare) webapp verwijst. Dit is gevaarlijk omdat de gebruiker er vanuit gaat dat alles in orde en veilig is omdat de link naar de officiële webapp gaat en dus is de kans groot dat het slachtoffer op de link klikt.

### 1.1.3 Stored XSS

Bij stored XSS wordt de JavaScript code opgeslagen op de server, vaak in een database. De code wordt dan automatisch uitgevoerd bij elke gebruiker die de opgeslagen content bekijkt of opvraagt. Deze kwetsbaarheid komt vaak voor bij gebruikers gegenereerde content zoals:

- Reacties, opmerkingen & reviews
- Forum & community berichten
- (Contact)formulieren & rich text editors
- ...

Figuur 1 uit onderdeel 1.1 toont hoe het plaatsen van een opmerking kan misbruikt worden, als er een XSS kwetsbaarheid aanwezig is. De aanvaller plaatst een opmerking met kwetsbare JavaScript i.p.v. een realistische opmerking. Deze opmerking en code worden opgeslagen in de database van de webapp. Andere gebruikers laden de pagina met opmerkingen en met de kwetsbare JavaScript, deze JavaScript wordt vervolgens uitgevoerd in de browser van het slachtoffer. Dit is gevaarlijk omdat de gebruiker zelf niks moet doen, het gebeurt gewoon.

## 1.2 Hoe XSS voorkomen?

### 1.2.1 Algemeen

XSS kwetsbaarheden komen vooral voor door een fout of onvoorzichtigheid in de code van de frontend maar dit wilt niet zeggen dat er bij het opzetten van de backend en de API daar geen aandacht aan besteed moet worden. Specifiek voor stored XSS is het beter om te voorkomen dat er überhaupt uitvoerbare JavaScript code kan opgeslagen worden in de database. Om dit te voorkomen is het dus belangrijk dat alle gebruikersinput die de API moet verwerken eerst gecontroleerd en gevalideerd wordt of deze geen gevaarlijke (JavaScript) payload bevat.

### 1.2.2 HTML escaping - HTML entity encoding

Een manier om de gebruikersinput veilig te maken op vlak van XSS is door gebruik te maken van HTML escaping of een andere benaming HTML entity encoding. Bij HTML escaping worden alle of bepaalde karakters vervangen door hun vastgelegde HTML entities. Zodat de tekst niet geïnterpreteerd kan worden als HTML code door een browser, waardoor de code niet uitgevoerd wordt.

[De lijst](#) met karakters en hun HTML entities geeft aan welke entity naam of entity nummer bij dat karakter hoort. Deze lijst werd vastgelegd door de Web Hypertext Application Technology Working Group (WHATWG). Bij HTML escaping worden niet altijd alle karakters vervangen door hun HTML entities, vaak worden gewone letters behouden en enkel de speciale of invloedrijke karakters vervangen.

Hier is een voorbeeld van HTML escaping: het karakter kleiner dan < wordt vervangen door zijn entity naam **&lt;** of door zijn entity nummer **&#60;**; de gebruiker krijgt wel gewoon het kleiner dan teken te zien op zijn/haar beeldscherm. Een voorbeeld van een gevaarlijke XSS payload die verandert door middel van HTML escaping: de payload **<img src=x onerror=alert(document.cookie)>** verandert dan in **&lt;img src=x onerror=alert(document.cookie)&gt;** waardoor het niet langer als HTML code wordt geïnterpreteerd maar gewoon als tekst en niet wordt uitgevoerd.

### 1.2.3 HTML Sanitization & HTML Stripping

Een andere manier om de gebruikersinput veilig te maken op vlak van XSS is door gebruik te maken van HTML Sanitization of HTML Stripping. Bij HTML Sanitization en HTML Stripping worden bepaalde of alle HTML tags verwijderd uit een opgegeven string of tekst. Door het verwijderen van HTML tags kan de gebruikersinput geen code meer bevatten en dus geen schade meer aanbrengen. Een voorbeeld van HTML Stripping is dat de string bestaande uit **<img src=x onerror=alert(document.cookie)>** gewoon volledig leeg wordt gemaakt omdat er geen tekst is buiten de HTML tag, er blijft een lege string over.

Het verschil tussen HTML Sanitization en HTML Stripping is dat bij sanitization worden enkel de niet tegenstaande HTML tags verwijderd, bij HTML Stripping worden alle HTML tags verwijderd uit een bepaalde tekst of string. Bij HTML Sanitization functies kan er dus een whitelist met toegestane HTML tags meegegeven worden, de tags in deze whitelist zullen dan niet verwijderd worden uit de tekst of string.



### 1.2.4 Cookies met HttpOnly attribuut

Als er wordt ingelogd met een account op een webapp wordt er vaak een sessie of access token aangemaakt en opgeslagen in de vorm van een cookie, deze is dan geldig voor een bepaalde tijdsperiode. Zodat de gebruiker tijdens deze tijdsperiode niet elke keer opnieuw zijn inloggegevens moet ingeven wanneer er een nieuwe API request gemaakt wordt. In plaats van de inloggegevens opnieuw mee te geven met de request wordt de cookie met de sessie of access token meegegeven.

Een cookie kan bepaalde attributen en eigenschappen hebben die vertellen aan de browser hoe de cookie moet gebruikt worden. Zo bestaat er een attribuut genaamd HttpOnly, als dit attribuut aanwezig is verteld het de browser dat deze cookie niet uitgelezen mag worden via JavaScript. Als er toch geprobeerd wordt om deze cookie uit te lezen via JavaScript, dan doet de browser alsof deze niet bestaat. Dit beschermt de cookie met de opgeslagen sessie of access token tegen token theft en session hijacking via XSS, omdat de cookie onbereikbaar is via JavaScript.

#### Figuur 3

Uitlezen van een cookie zonder HttpOnly attribuut doormiddel van JavaScript

```
> document.cookie
< 'access_token_unsafe=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwidXNlcm5hbWUiOiJJYW4iLCJyb2x1Ijo5LCJpYXQiOiJlE3NzE5NTIwNDIsImV4cCI6MTc3MTk1Mjk0Mn0.vZ8eVfT_V46aB_fUp2qs4QoK-VGp685e4iUe1R1J13g'
```

Zelfgemaakte afbeelding

#### Figuur 4

Uitlezen van een cookie met HttpOnly attribuut doormiddel van JavaScript

```
> document.cookie
< ''
```

Zelfgemaakte afbeelding

Figuur 3 en figuur 4 tonen het resultaat voor het opvragen van cookies met of zonder HttpOnly attribuut via JavaScript met de functie document.cookie. Deze functie werd uitgevoerd via de console van de DevTools in een Chromium browser.

## 1.3 Praktisch

Alle links naar het praktische deel van dit werk kunnen gevonden worden in de bijlagen van dit werk. Bij de webapps is er een pop-up systeem toegevoegd, deze geeft het succes of het mislukken van een request aan, dit zijn niet de JavaScript alert pop-ups. Figuur 7 toont hoe de JavaScript alert pop-ups (die aangeven dat XSS mogelijk is) eruitzien bij een Chromium browser.

### 1.3.1 Reflected XSS bij onveilige API

#### Figuur 5

Voorbeeld zoekfunctie met normale zoekterm

Post Search

Vul hieronder een zoekterm in om posts te zoeken die deze term bevatten.

Results for:

test

lan  
23-2-2026, 22:31:10

**Test**  
Dit is een test post om te kijken of alles werkt naar behoren.

Zelfgemaakte afbeelding

Het gebruikte praktische voorbeeld om reflected XSS aan te tonen voor dit onderzoek is een zoek functie waarmee een gebruiker kan zoeken naar bestaande posts, de gebruiker hoeft niet ingelogd te zijn. De zoekfunctie verwacht één of meerdere zoektermen en zal de zoekresultaten en de gebruikte zoekterm(en) terug geven aan de gebruiker. De zoek functie kan gebruikt worden via de webapp door onderaan de zoekbalk te gebruiken zoals in figuur 5 of door een query parameter te gebruiken in de URL, zoals hieronder:

[unsafe-app.ian-chains.be/?q=test](https://unsafe-app.ian-chains.be/?q=test)

#### Figuur 6

Voorbeeld zoekfunctie met XSS payload als zoekterm

Post Search

Vul hieronder een zoekterm in om posts te zoeken die deze term bevatten.

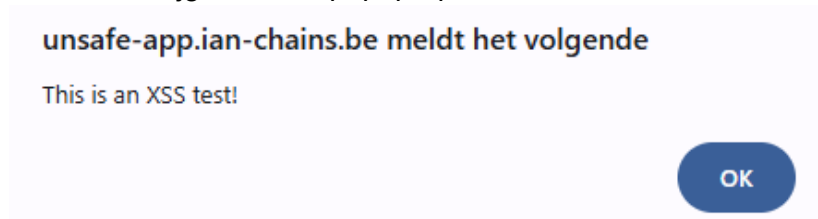
Results for:



Zelfgemaakte afbeelding

## Figuur 7

Gebruiker krijgt JavaScript pop-up in beeld



Zelfgemaakte foto

Figuur 6 toont de zoekresultaten van een zoekopdracht met de XSS payload `<img src=x onerror=alert('This is an XSS test!')">` als zoekterm. Deze zoekopdracht kan uitgevoerd worden door de payload in te vullen in de zoekbalk of door volgende URL te gebruiken: [unsafe-app.ian-chains.be/?q=%3Cimg%20src=x%20onerror=%22alert\(%27This%20is%20an%20XSS%20test!%27\)%22%3E](https://unsafe-app.ian-chains.be/?q=%3Cimg%20src=x%20onerror=%22alert(%27This%20is%20an%20XSS%20test!%27)%22%3E) Als deze zoekopdracht wordt uitgevoerd blijkt dat de JavaScript code `alert('This is an XSS test!')` ook effectief uitgevoerd wordt en de gebruiker krijgt een pop-up zoals in figuur 7 te zien. Deze webapp is dus kwetsbaar voor XSS aanvallen. In figuur 6 is te zien dat de payload als HTML code herkent en uitgevoerd wordt en er effectief geprobeerd wordt om de afbeelding in te laden maar doordat de afbeelding niet gevonden kan worden wordt het on error script uitgevoerd.

### 1.3.2 Stored XSS bij onveilige API

## Figuur 8

Aanmaken van een post met normale content

Post Creation

Vul de velden in om een nieuwe post aan te maken.

Zelfgemaakte afbeelding

## Figuur 9

Inladen van een post met normale content

Latest Posts

Bekijk hieronder de 5 laatst aangemaakte posts.

Ian  
24-2-2026, 17:57:01

**Welkom!**  
Welkom, dit is mijn eerste post op deze site!

Zelfgemaakte afbeelding

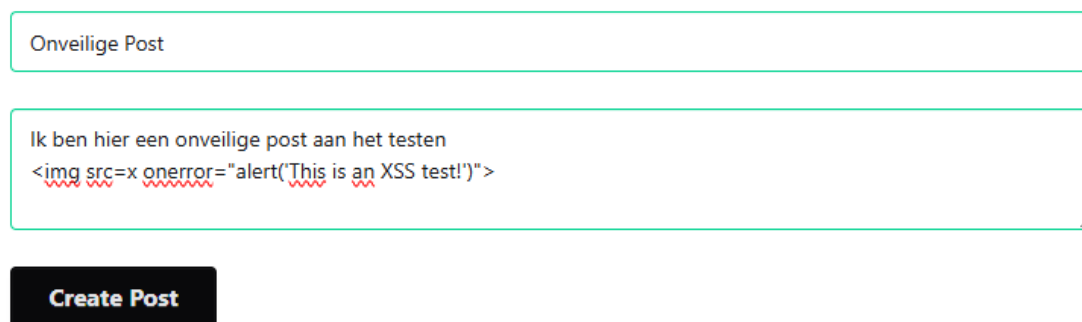
Het gebruikte praktische voorbeeld om stored XSS aan te tonen voor dit onderzoek is het kunnen aanmaken en opvragen van posts. Voor het aanmaken van posts moet een gebruiker ingelogd zijn met een account. Voor het inladen van posts worden telkens de 5 laatst aangemaakte posts weergegeven door te klikken op de load button. Figuur 8 toont hoe het aanmaken van een nieuwe post met normale content eruitziet. Figuur 9 toont hoe deze post met normale content eruitziet bij het inladen.

## Figuur 10

### Aanmaken van post met XSS payload als content

#### Post Creation

Vul de velden in om een nieuwe post aan te maken.



Onveilige Post

Ik ben hier een onveilige post aan het testen  
<img src=x onerror=alert('This is an XSS test!')>

Create Post

Zelfgemaakte afbeelding

## Figuur 11

### Inladen van post met XSS payload als content

#### Latest Posts

Bekijk hieronder de 5 laatst aangemaakte posts.



Ian  
24-2-2026, 18:02:20

Onveilige Post

Ik ben hier een onveilige post aan het testen

Zelfgemaakte afbeelding

Figuur 10 toont het aanmaken van een nieuwe post met zowel normale content als dezelfde XSS payload uit onderdeel 1.3.1. Figuur 11 toont hoe deze nieuwe post met XSS payload eruitziet bij het inladen, de normale content wordt op een normale manier afgebeeld maar de XSS payload wordt weer als HTML code geïnterpreteerd. Dezelfde JavaScript code wordt weer opnieuw uitgevoerd en de gebruiker krijgt opnieuw dezelfde pop-up te zien zoals in figuur 7.

### 1.3.3 Veilige API beschermt tegen XSS

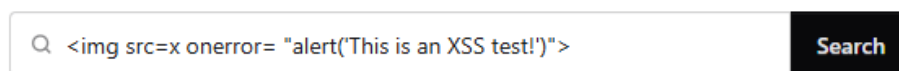
Om de API en het volledige webapp systeem te beschermen van XSS is er in dit onderzoek voor gekozen om HTML escaping of HTML entity encoding te gebruiken door middel van de escape functie van de [MarkupSafe](#) module in Python. Bij deze veilige API wordt er eerst HTML escaping uitgevoerd op alle gebruikersinput voordat het verder verwerkt wordt door het systeem.

#### Figuur 12

Voorbeeld veilige zoekfunctie met XSS payload als zoekterm

Post Search

Vul hieronder een zoekterm in om posts te zoeken die deze term bevatten.



A search bar with a magnifying glass icon on the left. Inside the bar is the text: `<img src=x onerror= "alert('This is an XSS test!')">`. To the right of the bar is a black button with the word "Search" in white text.

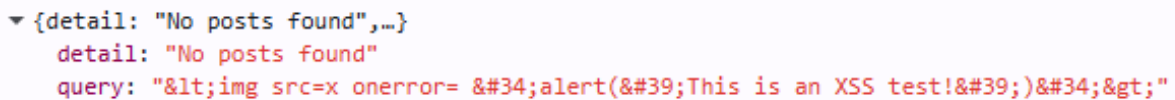
Results for:

`<img src=x onerror= "alert('This is an XSS test!')">`

Zelfgemaakte afbeelding

#### Figuur 13

Gebruikersinput waar HTML escaping op werd uitgevoerd, foto uit DevTools



```
▼ {detail: "No posts found",...}
  detail: "No posts found"
  query: "&lt;img src=x onerror= &#34;alert(&#39;This is an XSS test!&#39;)&#34;&gt;";
```

Zelfgemaakte afbeelding

Figuur 12 toont de veilige zoekopdracht van dezelfde XSS payload als uit onderdeel 1.3.1. Bij het uitvoeren van deze zoekopdracht krijgt de gebruiker geen pop-up in beeld, dus de XSS payload heeft niet gewerkt. Het verschil is duidelijk zichtbaar in de manier waarop de gebruikte zoekterm wordt afgebeeld, de zoekterm wordt niet meer afgebeeld als HTML code. Het is niet meer de foto die niet gevonden kan worden en een error oplevert, maar de gebruikte zoekterm wordt gewoon als tekst afgebeeld. Dit komt doordat de API de escaped versie van de gebruikte zoekterm teruggeeft aan de gebruiker waardoor het niet meer als HTML code kan geïnterpreteerd worden.

Voor het aanmaken en inladen van een post gebeurt net hetzelfde, de content van de post wordt eerst geëscaped voordat het wordt opgeslagen in de database, daardoor kan er geen enkele XSS payload opgeslagen worden in de database.

De cookie met de access token kreeg de extra attribuut `HttpOnly` toegewezen zodat de cookie niet meer uitleesbaar is via JavaScript, om op deze manier te beschermen tegen token theft via XSS.

## 2 CSRF - Cross-Site Request Forgery

### 2.1 Wat is CSRF?

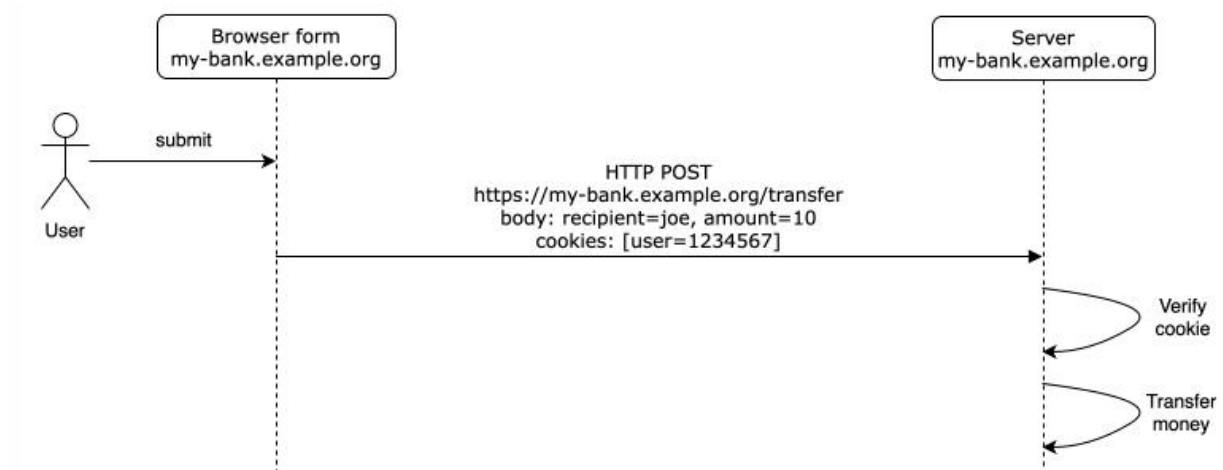
#### 2.1.1 Algemeen

CSRF is een beveiligingsprobleem waarbij het mogelijk is om ongewenste HTTP requests te versturen vanaf een andere site (cross-site) zonder dat de gebruiker dit wilde en meestal zonder enige actie uit te voeren. Door deze requests kan er data gedeeld of aangepast worden vanaf een andere site, zonder dat de gebruiker dit wilde en kan een request een kwaadaardig effect hebben.

#### 2.1.2 Voorbeeld Normaal Request

**Figuur 14**

Voorbeeld HTTP POST request, geld overschrijven



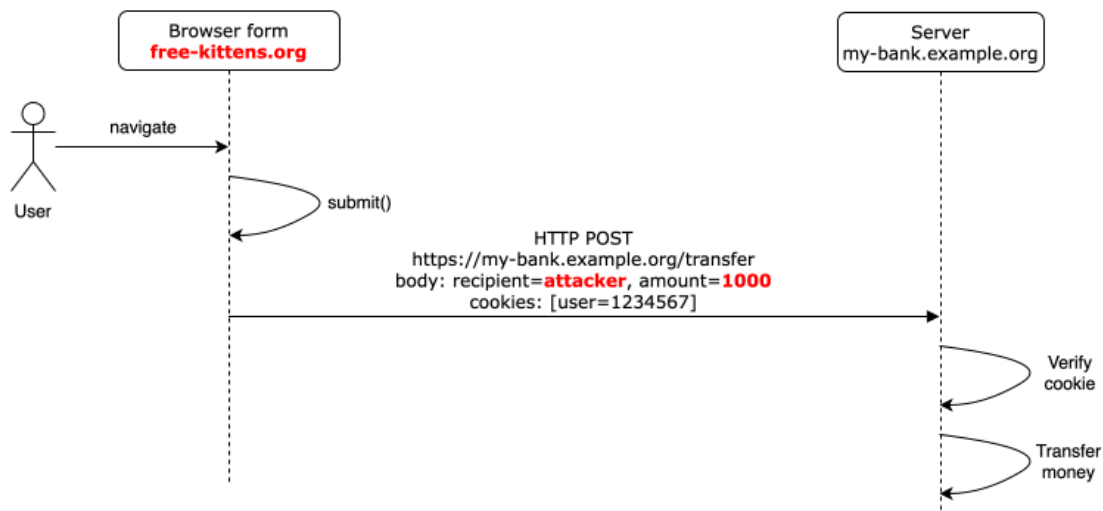
(Cross-site Request Forgery (CSRF) - Security | MDN, 2025)

Figuur 14 toont een POST request van een API waarbij de gebruiker zich op de webapp *my-bank.example.org* bevindt en geld wil overschrijven. De gebruiker voert de actie uit dus wordt er een HTTP POST request verstuurd. De gebruiker is reeds ingelogd en heeft dus een user UID of authenticatie token in de cookies van zijn browser opgeslagen. Deze cookies worden automatisch door de browser altijd meegegeven met een HTTP request. De server ontvangt deze request en controleert de user UID en/of de authenticatie token, als deze tokens geldig zijn dan wordt de request verwerkt en het geld overgeschreven.

### 2.1.3 Voorbeeld CSRF

#### Figuur 15

Voorbeeld CSRF bij POST request van figuur 14



(Cross-site Request Forgery (CSRF) - Security | MDN, 2025)

Figuur 15 toont dezelfde POST request als figuur 14 maar de gebruiker bevindt zich deze keer op een kwaadaardige webapp. Deze webapp zal zelf zonder enige aanleiding of actie een HTTP POST request versturen in naam van de gebruiker om geld over te schrijven. De browser herkent de hostname van de request en linkt deze aan de cookies van de bank webapp en geeft deze cookies en authenticatie tokens automatisch mee in de request. De server ontvangt deze request en zal deze verwerken als de authenticatie tokens nog geldig zijn. Een CSRF-aanval maakt dus gebruik van het feit dat een gebruiker reeds ingelogd is en authenticatie tokens reeds opgeslagen zijn als cookie, die worden dan automatisch meegegeven door de browser.

## 2.2 Hoe CSRF voorkomen?

### 2.2.1 SOP & CORS

Moderne browsers maken gebruik van de Same-Origin Policy (SOP) beveiligingsmaatregel om CSRF te voorkomen. SOP is een maatregel aan de clientside en zorgt ervoor dat er enkel gevoelige data tussen een frontend en API kan uitgewisseld worden als deze same-origin<sup>3</sup> zijn. Als er niet wordt voldaan aan de SOP dan wordt de response data van de API request geblokkeerd en kan de frontend deze data niet gebruiken/lezen. Zodat op deze manier een andere (kwaadaardige) webapp die cross-origin<sup>4</sup> is, geen gevoelige data kan lezen van een andere webapp.

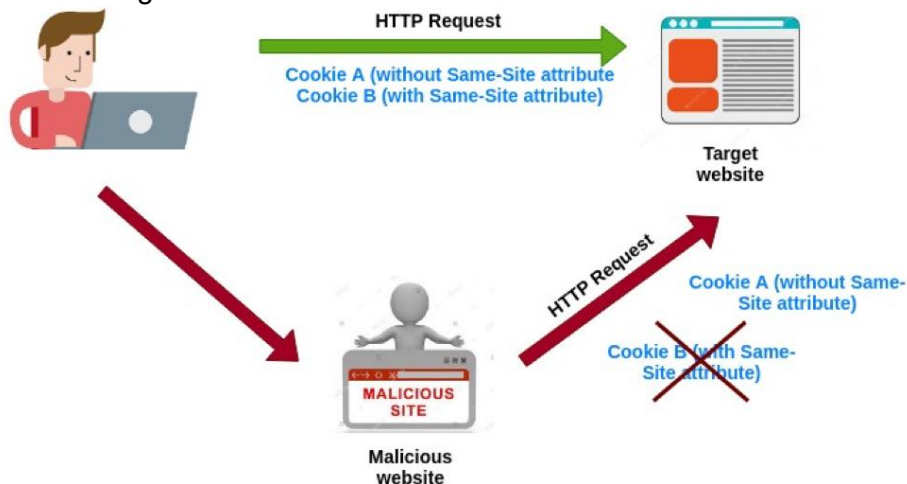
Cross-Origin Resource Sharing (CORS) is een uitzondering op SOP, met CORS kan een frontend of een andere webapp toegang krijgen om data te gebruiken van een andere API als deze cross-origin zijn. In de code van de API kunnen instellingen voor CORS worden toegevoegd zodat de cross-origin frontend toegang kan krijgen. De CORS instellingen zijn automatisch aanwezig als HTTP-header in elke response van de API.

De CORS instellingen van een API of webapp kunnen op 2 verschillende manieren opgevraagd worden, zodat de browser kan controleren of deze voldoen aan de eisen voor de HTTP-request die verstuurd zal worden. De manier die gebruikt wordt is afhankelijk van de originele request. Als de originele request een simple cross-origin request<sup>5</sup> is, dan wordt de originele request gewoon verstuurd en bij het verkrijgen van de response worden de CORS instellingen gecontroleerd. Als de originele request een non-simple cross-origin request is, dan zullen de instellingen eerst opgevraagd worden via een preflight request en zal de originele request enkel verstuurd worden als de CORS instellingen de controle doorstaan.

### 2.2.2 Cookies met SameSite attribuut

#### Figuur 16

Voorbeeld geblokkeerde cookie met SameSite attribuut



(SameSite Attribute Support in WSO2 Products, 2020)

<sup>3</sup> Webapps zijn **Same-Origin** aan elkaar als ze dezelfde protocol, hostname en poort hebben.

<sup>4</sup> Webapps zijn **Cross-Origin** als niet voldaan wordt aan alle eigenschappen van same-origin.

<sup>5</sup> De criteria waaraan een cross-origin HTTP-request moet voldoen om **simple** te zijn wordt opgesomd in de volgende bron: *Types of Cross-Origin HTTP Requests* - NI. (2023, February 17).

<https://www.ni.com/docs/en-US/bundle/g-web-development/page/types-of-cors-requests.html>



Cookies kunnen een SameSite attribuut met een bepaalde waarde toegewezen krijgen. Een cookie kan 3 verschillende waardes voor het SameSite attribuut toegewezen krijgen namelijk: none, lax of strict. Als de waarde strict is dan zal de cookie enkel meegeven worden wanneer de frontend en de API same-site<sup>6</sup> zijn, dit is de meest veilige keuze. Als de waarde none is dan zal de cookie altijd meegegeven worden dus ook wanneer de frontend en de API cross-site<sup>7</sup> zijn, dit is de minst veilige keuze. Als de waarde lax is dan zal de cookie enkel meegegeven worden wanneer het om een top-level navigation request gaat of wanneer de frontend en de API same-site zijn. Figuur 16 toont hoe een cookie met SameSite strict geblokkeerd wordt wanneer de origin cross-site is.

### 2.2.3 (Anti-)CSRF Tokens

(Anti-)CSRF tokens worden gebruikt om cross-site requests te detecteren en te blokkeren. De tokens werken op het principe dat enkel same-site requests de token kunnen uitlezen en meesturen met een request, doordat de browser via de Same-Origin Policy voorkomt dat externe webapps de pagina inhoud of cookies kunnen lezen. Wanneer de token ontbreekt of niet overeenkomt met wat er verwacht wordt, weet het systeem dat het om een cross-site request gaat en wordt deze geblokkeerd. CSRF tokens kunnen op verschillende manieren geïmplementeerd worden, maar voor dit onderzoek wordt gebruik gemaakt van een stateless signed CSRF token via de double submit methode.

Een stateless signed CSRF token is een token die stateless en gesigned is. Een token is stateless wanneer deze niet opgeslagen hoeft te worden en waarvan de geldigheid geverifieerd kan worden door een berekening. Deze token wordt beschermt met een handtekening en is dus signed, deze handtekening wordt gemaakt aan de hand van de token data zelf en een geheime sleutel. Als er geprutst wordt aan de token, zal de handtekening niet meer overeenkomen met de data en zal de token ongeldig worden.

De double submit methode is een manier om data aan te bieden, hierbij wordt zoals de naam het zelf zegt de data op 2 verschillende manieren aangeboden en verstuurd. De CSRF token wordt dus op 2 verschillende manieren opgenomen in een request, de token wordt opgenomen als cookie maar ook in een custom header. Bij de eerste verbinding met de webapp wordt de CSRF token aangevraagd. Bij die aanvraag wordt de token als cookie opgeslagen in de browser met SameSite strict en zonder HttpOnly attribuut zodat de JavaScript deze token kan uitlezen en kan opnemen in een custom header. Enkel requests vanaf een same-site origin kunnen deze cookie uitlezen en dus opnemen in de custom header. Wanneer de CSRF token niet op beide manieren is opgenomen in het request, of wanneer de waarden niet met elkaar overeenkomen, weet het systeem dat het om een cross-site request gaat.

---

<sup>6</sup> Webapps zijn **same-site** wanneer de apex domeinen van de webapps overeenkomen met elkaar, webapps met verschillende subdomeinen maar dezelfde apex domein zijn dus gewoon same-site.

<sup>7</sup> Webapps zijn **cross-site** wanneer er niet voldaan wordt aan alle voorwaarden van same-site.

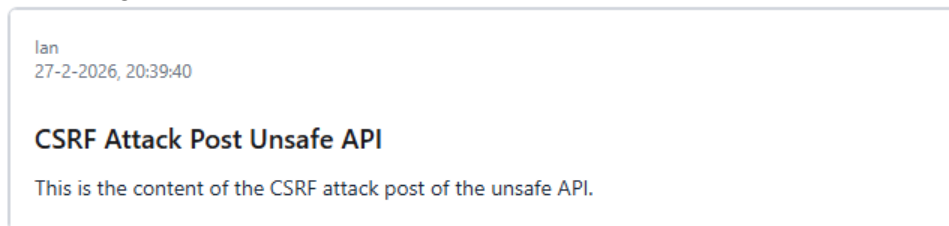
## 2.3 Praktisch

De frontend en de API (zowel voor de onveilige als veilige versie) van dit onderzoek zijn cross-origin aan elkaar doordat er andere subdomeinen worden gebruikt. Dus CORS moest ingesteld worden anders zou de data door de browser geblokkeerd worden, ook voor de onveilige API. Maar bij de onveilige API zijn de CORS instellingen verkeerd of niet veilig genoeg ingesteld want er wordt gewoon gebruik gemaakt van een wildcard waarmee alle cross-origin requests worden toegelaten.

### 2.3.1 CSRF bij onveilige API

#### Figuur 17

Post aangemaakt via CSRF



Zelfgemaakte afbeelding

#### Figuur 18

Gegevens van de CSRF request

| ▼ General               |                                                                                                                                                                                                      |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Request URL             | https://unsafe-api.ian-chains.be/posts                                                                                                                                                               |
| Request Method          | POST                                                                                                                                                                                                 |
| Status Code             | ● 201 Created                                                                                                                                                                                        |
| Remote Address          | [2606:4700:3032::6815:4cfe]:443                                                                                                                                                                      |
| Referrer Policy         | strict-origin-when-cross-origin                                                                                                                                                                      |
| ► Response headers (17) |                                                                                                                                                                                                      |
| ▼ Request Headers       |                                                                                                                                                                                                      |
| :authority              | unsafe-api.ian-chains.be                                                                                                                                                                             |
| :method                 | POST                                                                                                                                                                                                 |
| :path                   | /posts                                                                                                                                                                                               |
| :scheme                 | https                                                                                                                                                                                                |
| Accept                  | */*                                                                                                                                                                                                  |
| Accept-Encoding         | gzip, deflate, br, zstd                                                                                                                                                                              |
| Accept-Language         | nl-NL,nl;q=0.9,en-US;q=0.8,en;q=0.7                                                                                                                                                                  |
| Content-Length          | 104                                                                                                                                                                                                  |
| Content-Type            | application/x-www-form-urlencoded                                                                                                                                                                    |
| Cookie                  | access_token_unsafe=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxliwidXNlcm5hbWUiOiJYW4iLCJyb2xlljo5LCJpYXQiOiJlE3NzlyMjExMTQsImV4cCI6MTc3MjlyMjAxNH0.QD6WEa2VKxBRALEwQKGyLqIh75CZOCDwERbiAj3uP4 |
| Origin                  | https://attacker-unsafe.yourwebmaster.be                                                                                                                                                             |

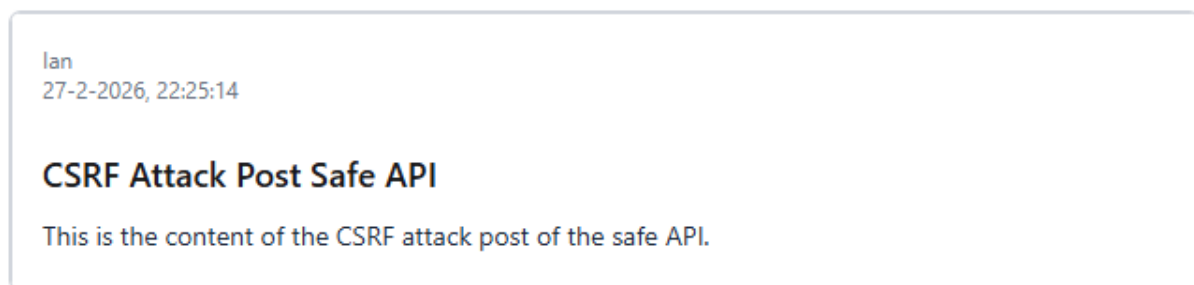
Zelfgemaakte afbeelding

Door de volgende link te openen wordt er automatisch een poging gedaan om een post aan te maken via CSRF: [attacker-unsafe.yourwebmaster.be/POST-Create-Post.html](https://attacker-unsafe.yourwebmaster.be/POST-Create-Post.html) Figuur 17 toont aan dat het gelukt is om via CSRF een post aan te maken, door het feit dat de gebruiker reeds ingelogd was en de access token nog steeds geldig was. Figuur 18 toont de informatie en gegevens aan van de API request die op de achtergrond gemaakt werd. De informatie verteld dat [attacker-unsafe.yourwebmaster.be](https://attacker-unsafe.yourwebmaster.be) de origin is van de request, de POST request naar de onveilige API succesvol was en dat de access token inderdaad opgenomen werd in de request.

### 2.3.2 CSRF ondanks correcte CORS configuratie

#### Figuur 19

Post aangemaakt via CSRF, ondanks correcte CORS configuratie



Zelfgemaakte afbeelding

#### Figuur 20

DevTools Console CORS Error



Zelfgemaakte afbeelding

Figuur 19 toont aan dat het gelukt is om via CSRF op de veilige API een post aan te maken, ondanks dat de CORS instellingen correct ingesteld zijn. Bij de CSRF aanval krijgt de gebruiker wel een CORS error in de console van de DevTools te zien. De response van de request wordt nu wel door de browser geblokkeerd maar dat heeft geen nut meer want de POST request is dan al verwerkt door de API. Dit komt doordat SOP een beveiligingsmaatregel is aan de clientside.

De aanmaak van deze post is gelukt doordat het hier om een simple cross-origin request gaat waarbij geen preflight gebruikt is om op voorhand de CORS instellingen te controleren. Daarbij staat de SameSite attribuut van de cookie met de access token nog altijd op none, door dit te veranderen naar strict zal de access token niet meer meegegeven worden bij cross-site requests waardoor deze CSRF aanval niet meer zou lukken.

### 2.3.3 (Anti-)CSRF Tokens beschermen veilige API

De SameSite attribuuft van de cookie met de access token werd aangepast naar strict om de eerder aangetoonde CSRF aanval niet meer mogelijk te maken. Maar om zeker volledig beschermd te zijn tegen CSRF aanvallen worden er stateless signed (Anti-)CSRF tokens via double submit gebruikt bij de veilige API.

### Figuur 21

## Gegevens van API request met CSRF token via double submit

| ▼ Request Headers  |                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| :authority         | safe-api.ian-chains.be                                                                                                                                                                                                                                                                                                                                                                                                            |
| :method            | POST                                                                                                                                                                                                                                                                                                                                                                                                                              |
| :path              | /posts                                                                                                                                                                                                                                                                                                                                                                                                                            |
| :scheme            | https                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Accept             | */*                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Accept-Encoding    | gzip, deflate, br, zstd                                                                                                                                                                                                                                                                                                                                                                                                           |
| Accept-Language    | nl-NL,nl;q=0.9,en-US;q=0.8,en;q=0.7                                                                                                                                                                                                                                                                                                                                                                                               |
| Content-Length     | 23                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Content-Type       | application/x-www-form-urlencoded                                                                                                                                                                                                                                                                                                                                                                                                 |
| Cookie             | csrf_token=.eJwFwQEBACAIA7AuJIDkonGAQ_8IbuO1XV7blqzslGA6l2VWwR3rfC2KUyUd5aC75zqlCw0lOT53VxMZ.aalUUQ_kvBE7tsrYsSnBtyVPlkSgh9j94; access_token_safe=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxliwidXNlcm5hbWUiOiJJYW4iLCJyb2xlljo5LjCjPjYXQiOiJlOT53VxMZ.aalUUQ_kvBE7tsrYsSnBtyVPlkSgh9j94; access_token=.eJwFwQEBACAIA7AuJIDkonGAQ_8IbuO1XV7blqzslGA6l2VWwR3rfC2KUyUd5aC75zqlCw0lOT53VxMZ.aalUUQ_kvBE7tsrYsSnBtyVPlkSah9j94 |
| Origin             | https://safe-app.ian-chains.be                                                                                                                                                                                                                                                                                                                                                                                                    |
| Priority           | u=1, i                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Referer            | https://safe-app.ian-chains.be/                                                                                                                                                                                                                                                                                                                                                                                                   |
| Sec-Ch-UA          | "Not:A-Brand";v="99", "Google Chrome";v="145", "Chromium";v="145"                                                                                                                                                                                                                                                                                                                                                                 |
| Sec-Ch-UA-Mobile   | ?0                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Sec-Ch-UA-Platform | "Windows"                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Sec-Fetch-Dest     | empty                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Sec-Fetch-Mode     | cors                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Sec-Fetch-Site     | same-site                                                                                                                                                                                                                                                                                                                                                                                                                         |
| User-Agent         | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36                                                                                                                                                                                                                                                                                                                   |
| X-Csrf-Token       | .eJwFwQEBACAIA7AuJIDkonGAQ_8IbuO1XV7blqzslGA6l2VWwR3rfC2KUyUd5aC75zqlCw0lOT53VxMZ.aalUUQ_kvBE7tsrYsSnBtyVPlkSah9j94                                                                                                                                                                                                                                                                                                               |

Zelfgemaakte afbeelding

Figuur 21 toont de gegevens van een normale API request voor het aanmaken van een post. Er is in de figuur te zien dat er effectief gebruik gemaakt wordt van de double submit voor de CSRF token. Dezelfde CSRF token is zowel opgenomen bij de cookies als onderaan in de custom CSRF header. De veilige API maakt dus nu gebruik van CSRF tokens om CSRF te voorkomen.

## Figuur 22

Set-cookie geblokkeerd wegens cross-site request

Set-Cookie: csrf\_token=.eJwFwQkBAcEIBMAuJhA5F4zDt\_0j3MyKuq70QmA3QCmWKeyTHT

This attempt to set a cookie via a Set-Cookie header was blocked because it had the "SameSite=Strict" attribute but came from a cross-site response which was not the response to a top-level navigation.

Zelfgemaakte afbeelding

Figuur 22 toont aan wat er gebeurt als er vanaf een cross-site toch geprobeerd wordt om een CSRF token aan te vragen en in te stellen als cookie. Zoals het script op deze pagina: [attacker-safe.yourwebmaster.be/GET-CSRF-Cookie.html](https://attacker-safe.yourwebmaster.be/GET-CSRF-Cookie.html) De browser herkent dat een cookie met SameSite strict ingesteld moet worden voor een totaal andere site dan de origin waardoor de request gemaakt werd, dus zal de browser deze cookie niet opslaan.

## Figuur 23

CSRF token ontbreekt bij request

| ▼ General               |                                        |
|-------------------------|----------------------------------------|
| Request URL             | https://safe-api.ian-chains.be/posts   |
| Request Method          | POST                                   |
| Status Code             | ● 403 Forbidden                        |
| Referrer Policy         | strict-origin-when-cross-origin        |
| ► Response headers (14) |                                        |
| ▼ Request Headers       |                                        |
| :authority              | safe-api.ian-chains.be                 |
| :method                 | POST                                   |
| :path                   | /posts                                 |
| :scheme                 | https                                  |
| Accept                  | */*                                    |
| Accept-Encoding         | gzip, deflate, br, zstd                |
| Accept-Language         | nl-NL,nl;q=0.9,en-US;q=0.8,en;q=0.7    |
| Content-Length          | 100                                    |
| Content-Type            | application/x-www-form-urlencoded      |
| Origin                  | https://attacker-safe.yourwebmaster.be |

Zelfgemaakte afbeelding

Figuur 23 toont aan dat een cross-site request waarbij de CSRF token ontbreekt niet werkt en een response met status 403 forbidden krijgt. Zoals de request die wordt gemaakt op deze pagina: [attacker-safe.yourwebmaster.be/POST-Create-Post-without-header.html](https://attacker-safe.yourwebmaster.be/POST-Create-Post-without-header.html)

## Figuur 24

Preflight door custom header

|         |            |           |                          |        |
|---------|------------|-----------|--------------------------|--------|
| ✖ posts | CORS error | fetch     | POST-Create-Post.html:30 | 0.0 kB |
| ✖ posts | 400        | preflight | Preflight                | 0.0 kB |

Zelfgemaakte afbeelding

Er zijn 2 redenen waarom het niet lukt om bij een cross-site request de custom header voor de CSRF token in te stellen. Zoals de cross-site request die uitgevoerd wordt op deze pagina: <https://attacker-safe.yourwebmaster.be/POST-Create-Post.html> De 1<sup>ste</sup> reden is dat de cross-site de cookie met de CSRF token niet kan uitlezen en deze dus ook niet kan opnemen in de custom header, als dit toch geprobeerd wordt zal de waarde van de custom header gewoon null zijn. De 2<sup>de</sup> reden is dat requests met een custom header non-simple cross-origin requests zijn, waardoor een preflight gebruikt wordt (zoals te zien is in figuur 24) en de browser ziet dat de attacker webapp niet opgenomen is in de CORS instellingen. Dus zal de request nooit verstuurd worden door de browser.

## Besluit

Het is duidelijk dat de beveiliging van API systemen zeer belangrijk is en geprioriteerd moet worden bij het uitwerken van een project. Om zo beveiligingsproblemen zoals Cross-Site Scripting en Cross-Site Request Forgery te voorkomen, anders kan dit catastrofale gevolgen hebben wanneer deze kwetsbaarheden misbruikt worden door mensen met slechte bedoelingen.

Er is ook één grote lijn en overeenkomst aanwezig tussen deze verschillende veelvoorkomende beveiligingsproblemen bij REST-API's. De directe input, gegevens en acties van een gebruiker kan nooit meteen vertrouwd worden en moet altijd eerst gecontroleerd en opgekuist worden voordat deze gebruikersinput en gegevens verwerkt worden. Zodat de input geen schadelijke payload of code bevat en geen schade kan aanbrengen aan het digitale systeem, de server, de databank of andere gebruikers.

De gebruikersinput kan veilig opgekuist worden door middel van HTML entity encoding of HTML sanitization om Cross-Site Scripting te voorkomen. De echtheid en afkomst van een request kan gecontroleerd worden door middel van (Anti-)CSRF tokens om zo Cross-Site Request Forgery te voorkomen.

Uit het onderzoek blijkt ook dat het correct configureren van een API systeem zeer belangrijk is om beveiligingsproblemen te voorkomen. Zo moet er voor gezorgd worden dat Cross-Origin Resource Sharing correct en specifiek ingesteld wordt en niet gewoon alles toegelaten wordt en dat cookies de juiste attributen toegewezen krijgen zodat ze op de juiste manier gebruikt worden door de browser.

## Bijlagen

Voor het praktische gedeelte van dit onderzoek is er gebruik gemaakt van een GitHub repository om de bestanden, de wijzigingen en documentatie overzichtelijk te kunnen bijhouden.

Alle bijlagen werden samen gebundeld tot een zip bestand. Het zip bestand werd samen met dit werk ingediend.

| Info & Onderwerp:                     | Bestandsnaam of link:                                                                                                       |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Referentielijst van dit onderzoek     | 25-26_GDC_Referentielijst_Ian_6ICWE.pdf                                                                                     |
| GitHub Repository                     | <a href="https://github.com/GO-atheneum-De-Tandem/GDC-Ian-2025-2026">github.com/GO-atheneum-De-Tandem/GDC-Ian-2025-2026</a> |
|                                       |                                                                                                                             |
| Onveilig API Systeem                  | <a href="https://unsafe-api.ian-chains.be">unsafe-api.ian-chains.be</a>                                                     |
| Onveilig API Systeem – OpenAPI Docs   | <a href="https://unsafe-api.ian-chains.be/docs">unsafe-api.ian-chains.be/docs</a>                                           |
| Onveilige Webapp (Frontend)           | <a href="https://unsafe-app.ian-chains.be">unsafe-app.ian-chains.be</a>                                                     |
|                                       |                                                                                                                             |
| Veilig API Systeem                    | <a href="https://safe-api.ian-chains.be">safe-api.ian-chains.be</a>                                                         |
| Veilig API Systeem – OpenAPI Docs     | <a href="https://safe-api.ian-chains.be/docs">safe-api.ian-chains.be/docs</a>                                               |
| Veilige Webapp (Frontend)             | <a href="https://safe-app.ian-chains.be">safe-app.ian-chains.be</a>                                                         |
|                                       |                                                                                                                             |
| CSRF Attacker Webapp op Onveilige API | <a href="https://attacker-unsafe.yourwebmaster.be">attacker-unsafe.yourwebmaster.be</a>                                     |
| CSRF Attacker Webapp op Veilige API   | <a href="https://attacker-safe.yourwebmaster.be">attacker-safe.yourwebmaster.be</a>                                         |

## Referentielijst

De referentielijst is opgenomen in een apart bestand om dit onderzoek overzichtelijk te houden. De referentielijst is ook gesorteerd per onderdeel van dit werk. Dit bestand kan je terug vinden in de meegeleverde bijlagen.

Het controleren van spelling van dit werkstuk is er gebruik gemaakt van [woordenlijst.org](https://woordenlijst.org) en [spelling.nu](https://spelling.nu).